

```

        {
            Console.WriteLine(
                "An exception occurred reading from the
file");
            return;
        }
        finally
        {
            if (sw != null) { sw.Close(); }
        }
    }
}

```

This becomes even more complicated if our finally block also has logic that may throw an exception or fail. What we want is a way to ensure that we always close resources when we are done with them and no later.

C/C++ handles this via a design pattern, resource allocation is initialization (RAII). Since local variables live on the stack and are automatically deallocated when the containing method returns, RAII utilizes them (and the deterministic destructors of C++) to automatically allocate resources and then deallocate them when the local variable goes away.

This idiom is essentially what the using declaration above provides.

```
using (StreamWriter sw = File.Create(file)) { }
```

declares a "disposable" variable `sw` that allocates a resource, in this case a file at the beginning of the using block and then releases that resource when `sw` goes out of scope at the end of the block.

In order to be the subject of a using block, the class must implement the `IDisposable` interface. The resulting code is called the Dispose Pattern. See the following MSDN link for more information:

<http://msdn.microsoft.com/en-us/library/fs2xkftw%28VS.80%29.aspx>

```

public class DisposePatternTest : IDisposable
{
    public DisposePatternTest()
    {
        Console.WriteLine("In DisposePatternTest ctor");
    }

    #region IDisposable Members

```